Выполнение лабораторной работы №3

Первоначальная настройка git.

Карпухин Клим

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**
3.1 Базовые сведения о Markdown 7
3.2 Обработка файлов в формате Markdown 10

4 Выполнение лабораторной работы **12**

5 Выводы **15**

Список литературы **16**

Список иллюстраций

4.1 Основная информация о лабораторной работе 12
4.2 Цель и задания лабораторной работы 12
4.3 Теоретическое введение лабораторной работы 13
4.4 Описание процесс выполнения лабораторной работы 13
4.5 Ответы на контрольные вопросы 14
4.6 Выводы лабораторной работы 14
4.7 Выводы лабораторной работы 14

Список таблиц

1 Цель работы

Цель данной лабораторной работы - научиться оформлять отчёты с помощью легковесного языка разметки Markdown.

2 Задание

- Создать отчёт по предыдущей лабораторной работе в формате Markdown.
- Предоставить отчёты в 3 форматах: `md`, `pdf` и `docx`

3 Теоретическое введение

3.1 Базовые сведения о Markdown

Чтобы создать заголовок, используйте знак (#), например:

```
# This is heading 1
## This is heading 2
### This is heading 3
#### This is heading 4
```

Чтобы задать для текста полужирное начертание, заключите его в двойные звездочки:

```
This text is **bold**.
```

Чтобы задать для текста курсивное начертание, заключите его в одинарные звездочки:

```
This text is *italic*.
```

Чтобы задать для текста полужирное и курсивное начертание, заключите его в тройные звездочки:

```
This is text is both ***bold and italic***.
```

Блоки цитирования создаются с помощью символа >:

```
> The drought had lasted now for ten million years, and the reign of
the terrible lizards had long since ended. Here on the Equator, in
the continent which would one day be known as Africa, the battle
for existence had reached a new climax of ferocity, and the victor
was not yet in sight. In this barren and desiccated land, only the
small or the swift or the fierce could flourish, or even hope to
survive.
```

Неупорядоченный (маркированный) список можно отформатировать с помощью звез- дочек или тире:

```
- List item 1
- List item 2
- List item 3
```

Чтобы вложить один список в другой, добавьте отступ для элементов дочернего списка:

```
- List item 1
 - List item A
 - List item B
- List item 2
```

Упорядоченный список можно отформатировать с помощью соответствующих цифр:

```
1. First instruction
1. Second instruction
1. Third instruction
```

Чтобы вложить один список в другой, добавьте отступ для элементов дочернего списка:

```
1. First instruction
 1. Sub-instruction
 1. Sub-instruction
1. Second instruction
```

Синтаксис Markdown для встроенной ссылки состоит из части `[link text]` , представляющей текст гиперссылки, и части `(file-name.md)` – URL-адреса или имени файла, на который дается ссылка:

```
[link text](file-name.md)
```

Markdown поддерживает как встраивание фрагментов кода в предложение, так и их размещение между предложениями в виде отдельных огражденных блоков. Огражденные блоки кода — это простой способ выделить синтаксис для фрагментов кода. Общий формат огражденных блоков кода:

1 `language 2 your code goes in here 3`

Верхние и нижние индексы: H_2O записывается как

```
H~2~O
```

2^{10} записывается как

```
2^10^
```

Внутритекстовые формулы делаются аналогично формулам LaTeX. Например, формула $\sin^2(x) + \cos^2(x) = 1$ запишется как

```
$\sin^2 (x) + \cos^2 (x) = 1$
```

Выключные формулы:

$$\sin^2(x) + \cos^2(x) = 1$$

{#eq:eq:sin2+cos2} со ссылкой в тексте «Смотри формулу ([**eq:eq:sin2+cos2**]).» записы- вается как

```
$$
\sin^2 (x) + \cos^2 (x) = 1
$$ {#eq:eq:sin2+cos2}

Смотри формулу ([-@eq:eq:sin2+cos2]).
```

3.2 Обработка файлов в формате Markdown

Для обработки файлов в формате Markdown будем использовать Pandoc https://pandoc.org/. Конкретно, нам понадобится программа `pandoc`, `pandoc-citeproc` https://github.com/jgm/pandoc/releases, `pandoc-crossref` https://github.com/lierdakil/pandoc-crossref/releases. Преобразовать файл `README.md` можно следующим образом:

```
pandoc README.md -o README.pdf
```

или так

```
pandoc README.md -o README.docx
```

Можно использовать следующий Makefile

```
FILES = $(patsubst %.md, %.docx, $(wildcard *.md))
FILES += $(patsubst %.md, %.pdf, $(wildcard *.md))

LATEX_FORMAT =

FILTER = --filter pandoc-crossref

%.docx: %.md
-pandoc "$<" $(FILTER) -o "$@"
```

```
%.pdf: %.md
-pandoc "$<" $(LATEX_FORMAT) $(FILTER) -o "$@"

all: $(FILES)
@echo $(FILES)

clean:
-rm $(FILES) *~
```

4 Выполнение лабораторной работы

Указал основную информацию о лабораторной работе(рис. 4.1).

```
author:
  name: Карпухин Клим
  degrees:
  orcid:
  email: 1032255588@rudn.ru
  affiliation:
    - name: Российский университет дружбы народов
      country: Российская Федерация
      postal-code: 117198
      city: Москва
      address: ул. Миклухо-Маклая, д. 6

## Title
title: "Выполнение лабораторной работы №2"
subtitle: "Первоначальная настройка git"
license: "CC BY"
```

Рисунок 4.1: Основная информация о лабораторной работе

Указал цель и задания лабораторной работы(рис. 4.2).

```
# Цель работы

- Изучить идеологиию и применение средств контроля версий.
- Освоить умения по работе с git

# Задание

- Создать базовую конфигурацию для работы с git
- Создать ключ SSH.
- Создать ключ PGP.
- Настроить подписи git.
- Зарегистрироваться на Github.
- Создать локальный каталог для выполнения заданий по предмету.
```

Рисунок 4.2: Цель и задания лабораторной работы

Написал теоретическое введение лабораторной работы(рис. 4.3).

```
# Теоретическое введение

Git - распределённая система контроля версий, предназначенная для
отслеживания изменений в файлах и организации совместной работы над
проектами. Для безопасного подключения к удалённым репозиториям,
например на Github, используются SSH-ключи, обеспечивающие
аутентификацию без ввода пароля. Для подтверждения авторства коммитов
применяются PGP-ключи (GPG), позволяющие подписывать изменения и
делать их проверяемыми. Регистрация на Github даёт возможность
удалённого хранения репозиториев и взаимодействия с другими
разработчиками. Локальный каталог для выполнения заданий
инициализируется как Git-репозиторий, в котором фиксируются все
изменения в ходе работы.
```

Рисунок 4.3: Теоретическое введение лабораторной работы

Описал процесс выполнения лабораторной работы(рис. 4.4).

```
## Установка программного обеспечения

Установил git и gh([рис. @fig-001]).

![Установка git и gh](image/1.jpg){#fig-001 width=70%}

## Базовая настройка Git

Задал имя и email репозитория, насторил utf-8 в выводе сообщений git,
задал имя начальной ветки, настроил параметры autocrlf и safecrlf
([рис. @fig-002]).

![ адал имя и email репозитория, насторил utf-8 в выводе сообщений
git, задал имя начальной ветки, настроил параметры autocrlf и
safecrlf](image/2.jpg){#fig-002 width=70%}
```

Рисунок 4.4: Описание процесс выполнения лабораторной работы

Ответил на контрольные вопросы(рис. 4.5).

```
# Контрольные вопросы

1.  **Системы контроля версий (VCS)** — это инструменты, отслеживающие изменения в
файлах. Они решают задачи сохранения истории проекта, возможности отката к старым
верс>
2.  **Хранилище** — это база данных проекта со всей историей. **Коммит** —
сохраненный снимок состояния файлов в этот момент. **История** — последовательность
таких сним>
3.  В **централизованных** VCS (например, SVN) история хранится на одном сервере. В
**децентрализованных** (Git, Mercurial) каждый разработчик имеет полную копию
истории>
4.  При единоличной работе вы инициализируете репозиторий, вносите изменения,
добавляете их в индекс (`add`), фиксируете (`commit`) и при необходимости
просматриваете ис>
5.  При работе с общим хранилищем сначала получают актуальные изменения (`pull`),
затем фиксируют свои наработки локально (`commit`), отправляют их на сервер (`push`)
и >
6.  Git управляет версиями, позволяет создавать и сливать ветки, гарантирует
целостность данных (через хэширование) и обеспечивает высокую скорость работы
благодаря лока>
7.  Основные команды: `init` (создать), `clone` (копировать), `add` (подготовить),
`commit` (сохранить), `status` (проверить), `log` (история), `pull` (забрать),
`push` >
8.  Локально: `git init` → `git add` → `git commit`. С удаленным: `git clone` → `git
push` (отправка) и `git pull` (получение обновлений).
9.  **Ветки** — это отдельные линии разработки. Они нужны, чтобы параллельно вести
новые функции или исправления ошибок, не затрагивая стабильную версию проекта до их
по>
10. Для игнорирования файлов (например, временных, с секретами или зависимостями)
используется файл `.gitignore`. Это делается, чтобы не засорять репозиторий мусором
```

Рисунок 4.5: Ответы на контрольные вопросы

Описал выводы к лабораторной работе(рис. 4.6).

```
# Выводы

В ходе лабораторной работы была выполнена базовая
настройка Git, сгенерированны SSH и PGP ключи для
аутентификации и подписи коммитов, а также была
произведена регистрация и настройка учётной
записи на Github. В результате настроено
автоматическое подписание коммитов, что
обеспечивает верификацию авторства, и созданно
рабочее пространство с использованием шаблона
репозитория, подготовленное для дальнейшего
выполнения заданий по предмету.
```

Рисунок 4.6: Выводы лабораторной работы

Скомпилировал отчёт в формат pdf(рис. 4.7).

```
kkarpuhin@kkarpuhin:~/Документы/lab2$ pandoc report_lab2.md -o report_lab2.pdf --pdf-engine=xelatex
 -V mainfont="Liberation Serif"
kkarpuhin@kkarpuhin:~/Документы/lab2$
```

Рисунок 4.7: Выводы лабораторной работы

5 Выводы

В ходе данной лабораторной работы я научился оформлять отчёты с помощью легковесного языка разметки Markdown.

Список литературы